

Primality tests

This article describes multiple algorithms to determine if a number is prime or not.

Trial division

By definition a prime number doesn't have any divisors other than 1 and itself. A composite number has at least one additional divisor, let's call it d . Naturally $\frac{n}{d}$ is also a divisor of n . It's easy to see, that either $d \leq \sqrt{n}$ or $\frac{n}{d} \leq \sqrt{n}$, therefore one of the divisors d and $\frac{n}{d}$ is $\leq \sqrt{n}$. We can use this information to check for primality.

We try to find a non-trivial divisor, by checking if any of the numbers between 2 and $\sqrt{n}$ is a divisor of n . If it is a divisor, then n is definitely not prime, otherwise it is.

```
bool isPrime(int x) {  
    for (int d = 2; d * d <= x; d++) {  
        if (x % d == 0)  
            return false;  
    }  
    return x >= 2;  
}
```

This is the simplest form of a prime check. You can optimize this function quite a bit, for instance by only checking all odd numbers in the loop, since the only even prime number is 2. Multiple such optimizations are described in the article about [integer factorization](#).

Fermat primality test

This is a probabilistic test.

Fermat's little theorem (see also [Euler's totient function](#)) states, that for a prime number p and a coprime integer a the following equation holds:

$$a^{p-1} \equiv 1 \pmod{p}$$

In general this theorem doesn't hold for composite numbers.